# HSD 4.2” 300x400 BWR
# 转图说明

**Ver 1.0**
**2022/07**

**DISCLAIMER: THIS DOCUMENT IS PROVIDED TO YOU "AS IS" WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE. SITRONIX AND THE AUTHORS OF THIS DOCUMENT DISCLAIM ALL LIABILITY, INCLUDING LIABILITY FOR INFRINGEMENT OF ANY PROPRIETARY RIGHTS, RELATING TO USE OR IMPLEMENTATION OF INFORMATION IN THIS DOCUMENT. THE PROVISION OF THIS DOCUMENT TO YOU DOES NOT PROVIDE YOU WITH ANY LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS**



Sitronix

# LIST OF CONTENT


**1 HSD Panel 4.2” 300x400 BWR ..... 3**

1-1. 填图相关寄存器设定 ..... 3

1-2. 填图格式与说明 ..... 3

1-3. 转图程序范例 ..... 5

**2 Revision History ..... 9**

# 1 HSD Panel 4.2” 300x400 BWR

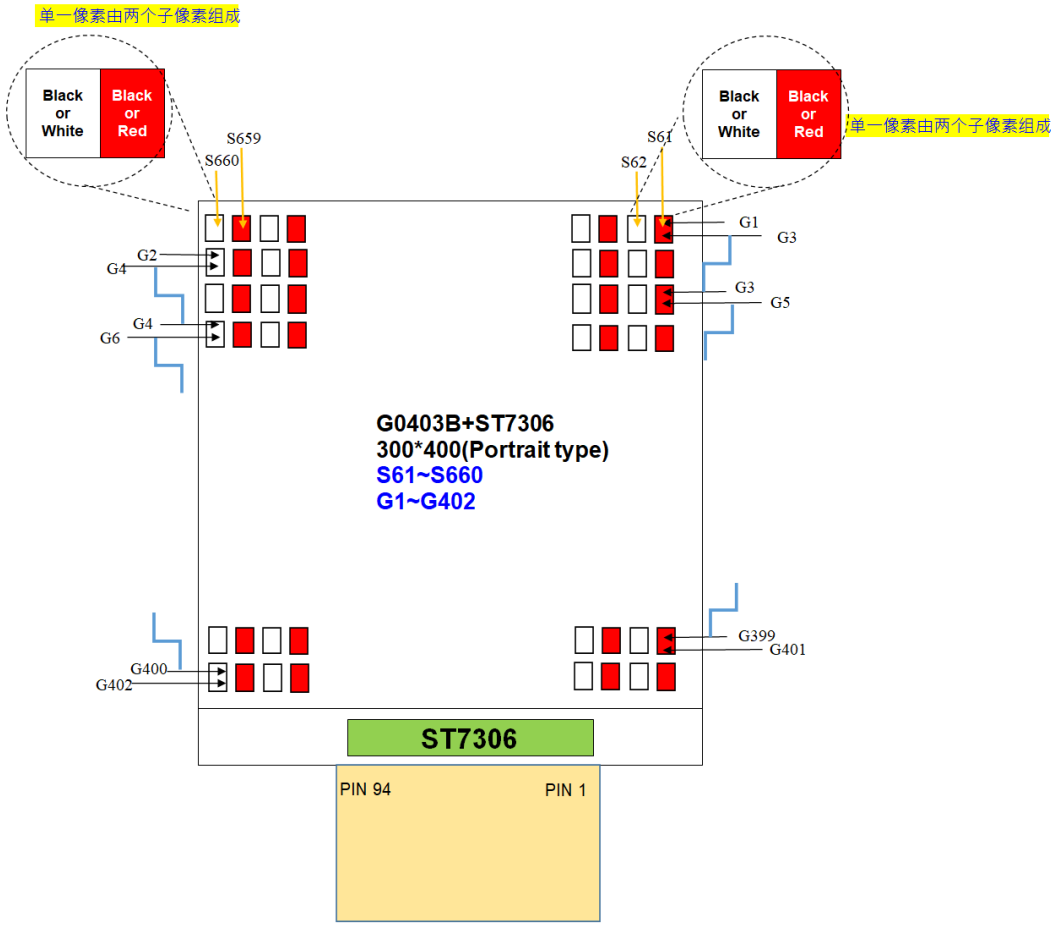
## 1-1. 填图相关寄存器设定

此模块填图相关寄存器设定如下表

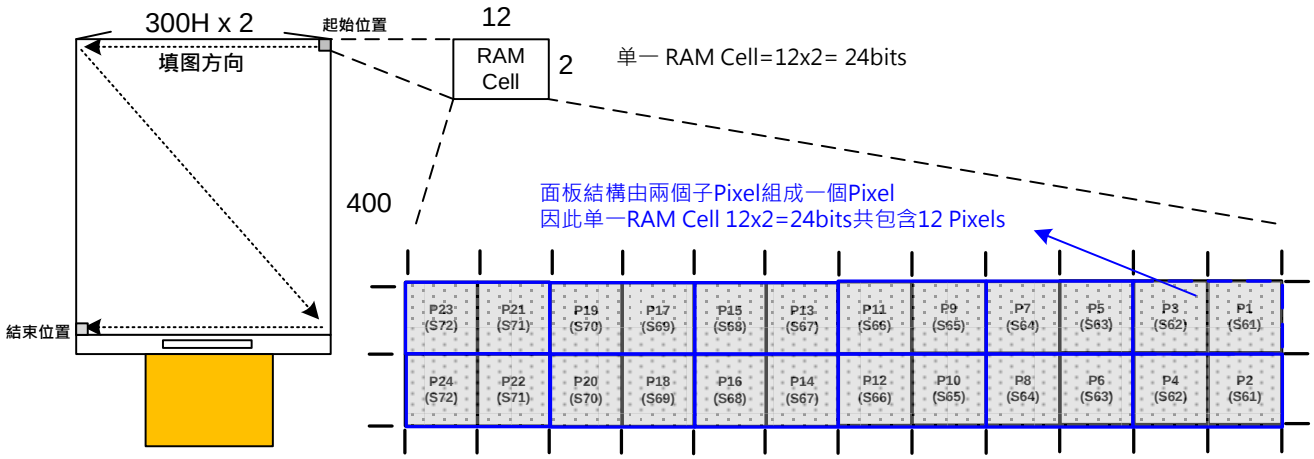
| Register Setting | Description |
|---|---|
| 0x36, cmd<br>0x00, data | Memory Data Access Control<br>MX/MY=0 ; DO=0 |
| 0x3A, cmd<br>0x11, data | Data Format Select<br>3write for 24 bit |

## 1-2. 填图格式与说明

HSD 4.2 吋 300Hx400V BWR 面板结构为单一像素由两个子像素组成，此两个子像素左右排列，并且可分别显示黑白或黑红去组成最后的像素，此 300Hx400V 的模块实际会使用到的 Source 数目为 300x2=600 根 (S61~S660)，如下图



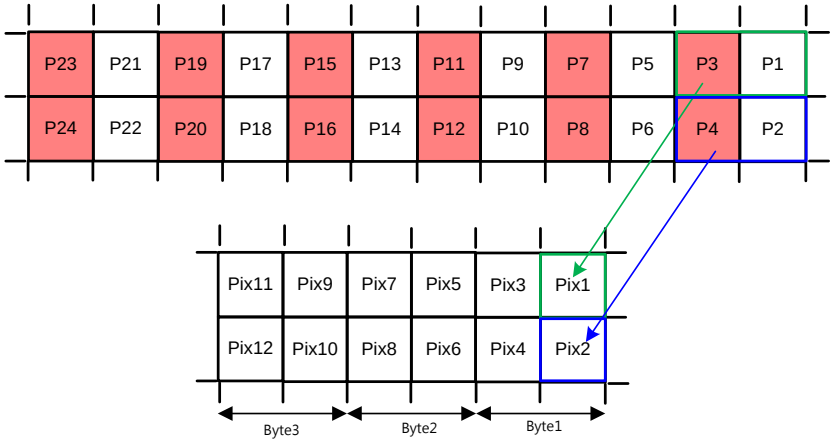
IC 的单位 RAM cell 结构为 12Hx2V，如果 X 方向分辨率不能被 12 整除，则填图时 X 方向最后一个 RAM cell 的资料仍须完整填写才能出现显示．因模块 X 方向共 300Hx2=600 sub-pixels 可以被 12 整除，所以不需考虑 X 方向分辨率不能被 12 整除的情况．




依照寄存器设定，数据输入格式为3bytes写入24bit数据，如下 (设定BPS=1" of command 0x3Ah)

| Command | A0 | D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|---|---|---|---|---|---|---|---|---|---|
| DDRAM write | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 1st write | 1 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 |
| 2nd write | 1 | P9 | P10 | P11 | P12 | P13 | P14 | P15 | P16 |
| 3rd write | 1 | P17 | P18 | P19 | P20 | P21 | P22 | P23 | P24 |

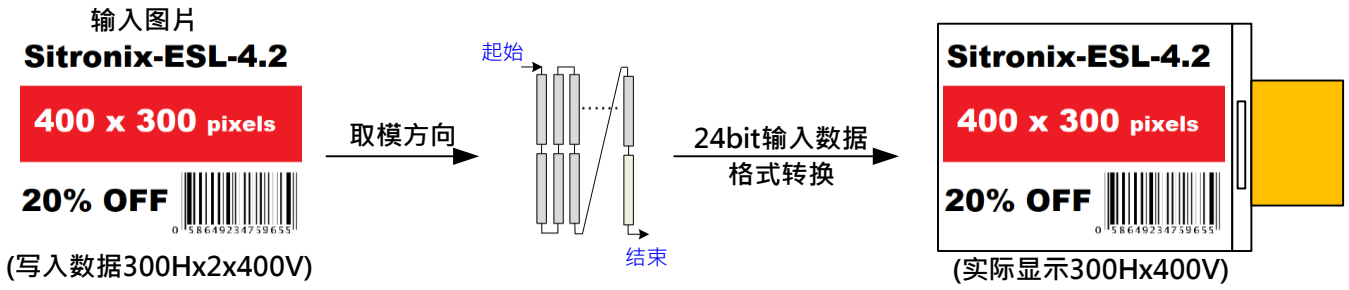
写入24bit数据对映到黑白紅显示结果如下(设定"MC8=0" of command 0x3Ah)
此面板结构为由两个子像素组成一个显示像素，所以单一RAM cell总共包含12个显示像素．

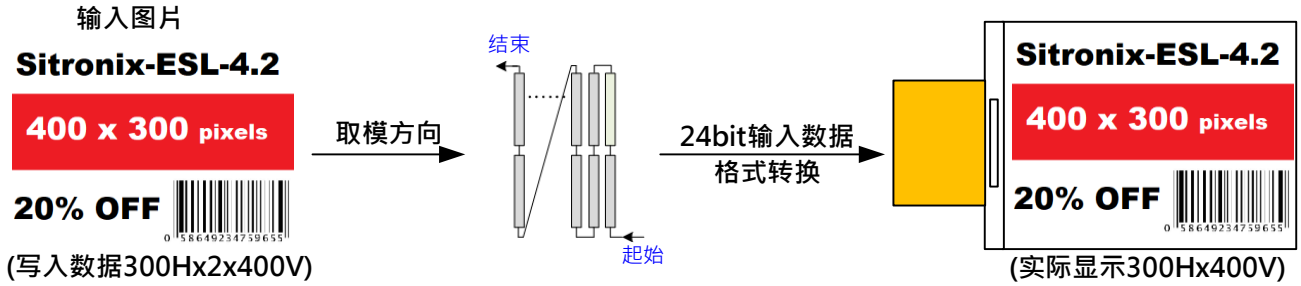



显示像素组成如下
P1+P3=>Pix1
P2+P4=>Pix2
P5+P7=>Pix3
P6+P8=>Pix4
P9+P11=>Pix5
P10+P12=>Pix6
P13+P15=>Pix7
P14+P16=>Pix8
P17+P19=>Pix9
P18+P20=>Pix10
P21+P23=>Pix11
P22+P24=>Pix12

## 1-3. 转图程序范例

图片取模方向请参考下图，实际上写入数据量为 300x2x400bits (共 30000bytes) . 对图片进行取模时，IC 置于模块不同位置，取模方向也需对应跟着改变，不同取模方向与显示结果请参考下图



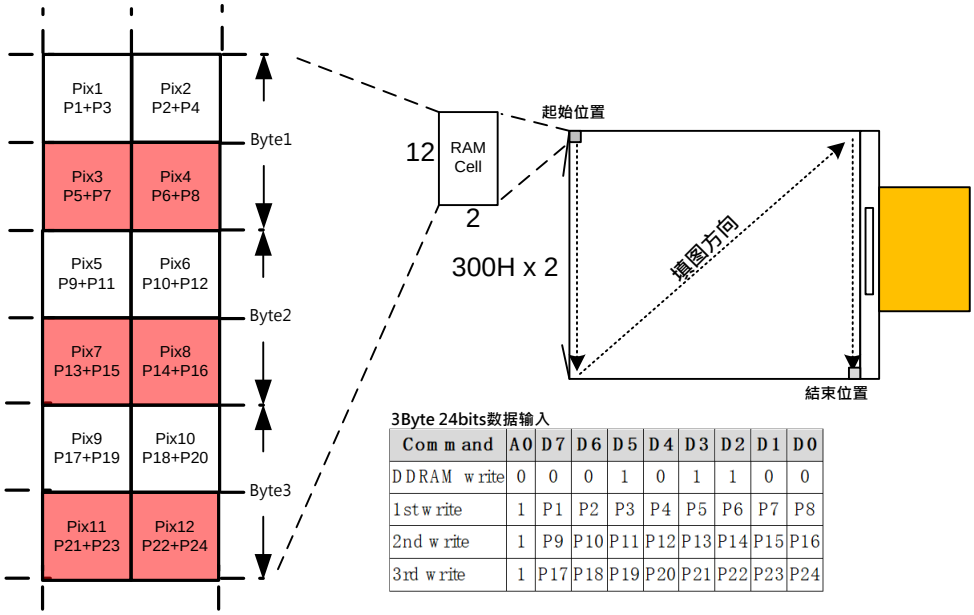
## 1-3-1 填图范例

```
#define XSIZE          (300x2)              //面板结构为两个子像素组成单一像素
#define YSIZE          (400)
#define COL_NUM        (XSIZE / 12)
#define ROW_NUM        (YSIZE / 2)
/*
每个列地址，一次写入 3 个字节 24BIT，显示 2 行 12 列，
0x05~0x36，50 个地址，按照 50x12=600，每个地址 3 个字节
每个行地址，一个地址包含 2 行，400 行，200 个地址，0~199
*/

void RAM_refresh(void)
{
    uint16_t row,col;
    uint8_t b[3];

    write_cmd(0x2A);
    write_dat(0x05);
    write_dat(0x36);
    write_cmd(0x2B);
    write_dat(0x00);
    write_dat(0xC7);
    write_cmd(0x2C);
    for (row = 0; row < ROW_NUM; row++)
    {
        for (col = 0; col < COL_NUM; col++)
        {
            fill_cell_r270(row, col, &b[0], &b[1], &b[2]);      //参考 1-3-2_24bit 数据格式转换
            _send_bytes(b, 3);
        }
    }
}
```

## 1-3-2 24bit 输入数据格式转换




3Byte 24bits数据输入

| Command | A0 | D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|---|---|---|---|---|---|---|---|---|---|
| DDRAM write | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 |
| 1st write | 1 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 |
| 2nd write | 1 | P9 | P10 | P11 | P12 | P13 | P14 | P15 | P16 |
| 3rd write | 1 | P17 | P18 | P19 | P20 | P21 | P22 | P23 | P24 |

```
#define PIXEL_W_BIT             (0x00)
#define PIXEL_B_BIT             (0x03)
#define PIXEL_R_BIT             (0x02)
static void fill_cell_r270(uint32_t row, uint32_t col, uint8_t *byt1, uint8_t *byt2, uint8_t *byt3)
{
    static uint8_t b1,b2,b3;
    static uint16_t x,y;
    x = row* 2;
    y = col * 6;                //单一画素由两个子画素构成

    b1 = 0;   //默认白色
    if (get_pixel(x + 0, y + 0)== PIXEL_B_BIT){b1 |= 0xA0;}
    if (get_pixel(x + 1, y + 0) == PIXEL_B_BIT){b1 |= 0x50;}
    if (get_pixel(x + 0, y + 1) == PIXEL_B_BIT){b1 |= 0x0A;}
    if (get_pixel(x + 1, y + 1) == PIXEL_B_BIT){b1 |= 0x05;}
    if (get_pixel(x + 0, y + 0)== PIXEL_R_BIT){b1 |= 0x80;}
    if (get_pixel(x + 1, y + 0) == PIXEL_R_BIT){b1 |= 0x40;}
    if (get_pixel(x + 0, y + 1) == PIXEL_R_BIT){b1 |= 0x08;}
    if (get_pixel(x + 1, y + 1) == PIXEL_R_BIT){b1 |= 0x04;}
    *byt1 = b1;
```

```
    b2 = 0;   //默认白色
    if (get_pixel(x + 0, y + 2) == PIXEL_B_BIT){b2 |= 0xA0;}
    if (get_pixel(x + 1, y + 2) == PIXEL_B_BIT){b2 |= 0x50;}
    if (get_pixel(x + 0, y + 3) == PIXEL_B_BIT){b2 |= 0x0A;}
    if (get_pixel(x + 1, y + 3) == PIXEL_B_BIT){b2 |= 0x05;}
    if (get_pixel(x + 0, y + 2) == PIXEL_R_BIT){b2 |= 0x80;}
    if (get_pixel(x + 1, y + 2) == PIXEL_R_BIT){b2 |= 0x40;}
    if (get_pixel(x + 0, y + 3) == PIXEL_R_BIT){b2 |= 0x08;}
    if (get_pixel(x + 1, y + 3) == PIXEL_R_BIT){b2 |= 0x04;}
    *byt2 = b2;

    b3 = 0;   //默认白色
    if (get_pixel(x + 0, y + 4) == PIXEL_B_BIT){b3 |= 0xA0;}
    if (get_pixel(x + 1, y + 4) == PIXEL_B_BIT){b3 |= 0x50;}
    if (get_pixel(x + 0, y + 5) == PIXEL_B_BIT){b3 |= 0x0A;}
    if (get_pixel(x + 1, y + 5) == PIXEL_B_BIT){b3 |= 0x05;}
    if (get_pixel(x + 0, y + 4) == PIXEL_R_BIT){b3 |= 0x80;}
    if (get_pixel(x + 1, y + 4) == PIXEL_R_BIT){b3 |= 0x40;}
    if (get_pixel(x + 0, y + 5) == PIXEL_R_BIT){b3 |= 0x08;}
    if (get_pixel(x + 1, y + 5) == PIXEL_R_BIT){b3 |= 0x04;}
    *byt3 = b3;

}
```

# 2 Revision History

| Version | Date | Description |
|---|---|---|
| V1.0 | 2022/07 | ● Preliminary. |
| | | |
| | | |